

Homework 1

9/14/2026

100 Points Possible

Attempt 1



In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

▼ Details



Assignment Overview - Making Progress

Skills you will learn

- Using various Unix tools to build an OS
- Using the qemu emulator and the Gnu debugger to run and debug your OS
- Compiling your OS
- Practice ARMv8 assembler by calling functions to write to the screen
- Calling an assembly function from C, and calling C from assembly

Things you will need

- Install course software if you haven't already. [Link to Course Software](https://canvas.tntech.edu/courses/6565/pages/install-virtual-machine-for-the-course) (<https://canvas.tntech.edu/courses/6565/pages/install-virtual-machine-for-the-course>) installation instructions.
- [libos.a](https://canvas.tntech.edu/courses/6565/files/1145683?wrap=1) (<https://canvas.tntech.edu/courses/6565/files/1145683?wrap=1>)
- [boot.S](https://canvas.tntech.edu/courses/6565/files/1145718?wrap=1) (<https://canvas.tntech.edu/courses/6565/files/1145718?wrap=1>)
- [libos.h](https://canvas.tntech.edu/courses/6565/files/1145692?wrap=1) (<https://canvas.tntech.edu/courses/6565/files/1145692?wrap=1>)

Introduction

Your first assignment will use a library provided by the professor to create simple program that will boot in qemu and boot message on the screen. This program will represent your initial code for your "operating system" (OS). To get credit for this assignment, you must follow the specification given, including the function prototypes and the instructions of which languages to use for which functions.

Tools that you will need

You will need to use the following tools to complete your assignment, and **all of them are provided as part of the course virtual machine**: You will need to download and install VirtualBox. However, for the rest, you do **not** need to download, install, or set them up. Their descriptions below are just to familiarize you with their function.

- **Qemu**, an aarch64 emulator. You will use this emulator to run, and test your operating system.
- **gdb**, the Gnu Debugger. If you are using an Intel machine for your assignments, then you will use **gdb-multiarch** to debug your operating system.
- **ddd**, the Data Display Debugger. This program provides a graphical user interface to GDB. It has some nice features that you will not find in other debuggers, even though the interface is old.

- **gcc**, the Gnu C compiler. If you are using an Intel machine for your assignments, then you will use the cross compiler **aarch64-linux-gnu-gcc**. The Gnu C compiler will generate object code for the main driver of your stage 2 operating system. You will run gcc (or aarch64-linux-gnu-gcc) to both compile C code as well as ARMv8 assembler code.
- **Debian**, a distribution of Linux. You can use another distribution if you wish. You will, however be on your own in getting things to work.
- **VirtualBox**, a free but commercial grade emulator. You will use VirtualBox only if you are using an Intel machine. It will allow you to run Debian Linux under Windows, Mac OS X, or another distribution of Linux without having to install Linux on your hard drive in a separate partition. At the end of the class, if you do not want to keep Linux, you can simple delete the virtual machine and uninstall VirtualBox.
- **UTM**, a free emulator. You will use UTM only if you are using an ARM based Mac. It will allow you to run Debian Linux under Mac OS X without having to install Linux on your hard drive in a separate partition. At the end of the class, if you do not want to keep Linux, you can simple delete the virtual machine and uninstall UTM.

Creating Your OS

In order to create your operating system you will need to write a simple main driver in C. The pseudo-code for the main is:

```
int main(void) {
    call boot_win function // see description below
    loop forever
        loop from perc from 10 to 100 in steps of 10
            call print_progress function with row 19, column 4, message "Progress: ", percent perc, and width 10
            call wait_msec function with argument 500000;
        return 0;
}
```

The main() driver of your OS first calls boot_win() to display a boot status. For now, the boot status is fake as you have not built the parts of the OS that the status message represents, but you will build these components in later assignments. Then the main() enters a loop that prints a (fake) progress bar. The progress bar starts with a message and then prints a horizontal bar made up of equal signs. Below is an example of what will appear on the screen.

```
+-----+
|
|  [OK] Console
|  [-] Memory
|  [-] Kernel
|  [-] Interrupts
|  [-] Processes
|
|  Memory: =====
|
+-----+
```

You will write boot_win() in C and print_progress() in ARMv8 assembly. The function wait_msec() is provided for you in the libos.a library that you will download from the link given in the above "Things you will need" section. The function prototypes are given in the table below. Make sure that you follow them. **You must write the function print_progress() in ARMv8 assembly and you must write the main() in C in order to get credit for this assignment.** You do not need to write the print_to() function or the putc_to() function (described below). The print_to() and putc_to() functions are provided for you in the libos.a library.

Box Drawing API

Function prototype	Description
void boot_win();	Prints the boot message window. This function must be written in C.

<pre>int print_progress(unsigned int row, unsigned int column, const char *message, unsigned int percent, unsigned int width);</pre>	The function print_progress() draws a progress bars. The progress bar consists of a message followed by a sequence of equal signs. The number of equal signs is a percentage of the width parameter. For example, if message is "Progress: ", percent is 50, and width is 10, then the following will be printed: Progress: ===== It calls strlen() to calculate the length of the progress message. It calls print_to() to print that progress message, and calls putc_to() in a loop to print the progress bar made up of equal signs ("=").
<pre>int strlen(const char *s)</pre>	The function calculates the length of the of the string by counting number characters in the string. It stops counting when it reads a zero byte, and returns the string's length. You must write this function in C/C++.

The pseudocode for the print_progress function is as follows:

```
print_progress:
/* row is in          w0
   column is in       w1
   address of label is in x2
   percent is in      w3
   width is in        w4 */
save x29 and x30
save any registers from x19 to x28 that you may use as local variables
print the message using print_to() with argument for row, column, and label address
if percent is above 100 then return 1 /* error, called with bad parameters */
calculate the the number of '=' to draw using mul and udiv /* (percent x width)/100 */
/* leave answer is a register acting as a local variable, say x19 */
call strlen() on label address (put in a register, say x20 */
put '=' in a local variable (say x21)
loop
    if count of equal signs (in x19) is zero
        put ' ' (space) in local variable (x21)
    if width is 0
        return (i.e. jump to .return_ok)
    call putc_to to print the "=" (or the space) at column + length of label string (in x20)
    increment column by 1
    decrement count of equal signs by 1 (x19)
    decrement width by 1
end loop
.return_with_error:
    mov w0, 1
    b     done
.return_ok:
    mov w0, 0
.done
```

```
clean up the stack
ret
```

Put the code for the main, strlen(), boot_win(), and what other functions you may write in C in the file kernel.c. Place the print_progress() function in a file name progress.S.

Building Your OS

You will create a Makefile to build your OS. The following is the output of the professor's makefile when he builds his OS (with comments):

```
# NOTE: for Mac M1, M2, or M3 users, remove the "aarch64-linux-gnu-"
# from each of the commands below.
# compile kernel.c into kernel.o
aarch64-linux-gnu-gcc -g -pedantic -Wall -Wextra -fPIC -std=gnu2x -MMD -c kernel.c -o kernel.o
# assemble boot.S into boot.o
aarch64-linux-gnu-gcc -g -MMD -c boot.S -o boot.o
# assemble progress.S into progress.o
aarch64-linux-gnu-gcc -g -MMD -c progress.S -o progress.o
# link kernel.o, boot.o, progress.o, and libos.a together to make kernel.elf
aarch64-linux-gnu-ld -g -N -Ttext=0x10000 -o kernel.elf kernel.o boot.o progress.o libos.a
# Note that the linker may give the following complaint. Ignore it.
# aarch64-linux-gnu-ld: warning: kernel.elf has a LOAD segment with RWX permissions
```

Your Makefile should compile your OS similarly. Make sure you write a "good" makefile. In other words, you should set up your dependencies such that no unnecessary compiling occurs. **If you do not write a correct Makefile and do not implement the dependencies to reflect the dependency graph below, then you will get a zero on this assignment!** A working Makefile should, if you change your assembly language file, compile only the assembly file but not your C file. Figure 2 shows the dependency tree that your Makefile should implement.

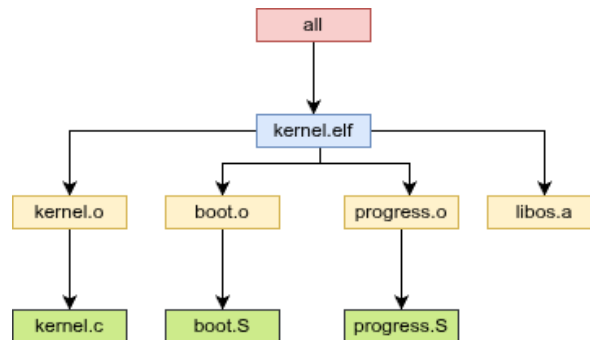


Figure 2: Makefile dependency graph. The root target is all. All has one child, kernel.elf. kernel.elf has four children: kernel.o, boot.o, progress.o, and libos.a. kernel.o has a child kernel.c. boot.o has a child boot.S. progress.o has a child progress.S. libos.a has no children.

Include a target called "clean" in your makefile. You should be able to type "make clean" to delete all of your object files, files with the ".d" extension, and your OS binary (in other words, only the .c and .S source files, libos.a, and Makefile should be left). Also, include a target for run and debug (see below) in your makefile.

Running Your OS

Now you can run your OS and see if it works. At the Linux prompt, issue the following command:

```
qemu-system-aarch64 -machine raspi3b -kernel kernel.elf
```

Debugging Your OS

To debug your OS, you will need to start Qemu in debug mode by adding both the -S and -s options as follows.

```
qemu-system-aarch64 -machine raspi3b -S -s -kernel kernel.elf
```

Qemu will launch and then immediately halts at the very first instruction. At this point, Qemu is waiting for you to attach to it with the GDB debugger. So, open another Linux terminal and enter the following command:

```
# if running on Intel hardware:
ddd --debugger 'gdb-multiarch' kernel.elf
# if running on ARM Mac:
ddd kernel.elf
```

Next, at the gdb prompt, enter the following:

```
target remote localhost:1234
```

Make your debugging life super easy by using the following as a rule in your makefile. This rule, when you type "make debug" will automatically start qemu, start the debugger, connect to qemu, and set a breakpoint on main():

```
debug:
    qemu-system-aarch64 -machine raspi3b -S -s -kernel kernel.elf &
    # Note that if you are running on an ARM Mac, substitute "gdb-multiarch" with "gdb".
    ddd --debugger 'gdb-multiarch' -ex "target remote localhost:1234" -ex "break main" -ex "continue" kernel.elf
```

Now, at the gdb prompt you can enter a number of debugging commands that allow you to look at registers, memory, single step through the code (for both the assembly and C), continue booting, etc. You should reference the gdb manual found at [Gnu Debugger URL \(http://sourceware.org/gdb/current/onlinedocs/gdb/\)](http://sourceware.org/gdb/current/onlinedocs/gdb/). DDD is just a graphical user interface on top of gdb. With DDD, you can single step, step over, and use other debugging functions by simply pressing buttons, and you can hover over variables to see their values in balloon pop-ups. If DDD does not support any gdb command in its menus, then you can type the command at the gdb prompt at the bottom of the graphical interface.

In summary, your Makefile should include three targets to clean, run and debug your OS, one called "clean", one called "run", and one called "debug". The professor should be able to type "make clean" to remove all .o and .elf files. The professor should be able to type "make debug" to start Qemu and DDD to debug your program. Finally, if the professor wants to watch your OS without pause (without it going into debugging mode), he should be able to type "make run" to run Qemu, *without* the -S and -s options, and watch your OS boot up and print its message.

A video of how the finished product should look, including how the Makefile should behave can be found at the following link:

Figure 3: The above is a video showing the instructor typing commands in a terminal. The terminal prompt shows that he is in his project 1 source directory. He clears the screen by typing "clear". Then he enters the command "make". The commands to create the operating systems files appear on the screen. Finally, the types "make run" and another screen appears showing VM booting, clearing the screen, and then printing a box on the screen with the boot text and a progress bar.

 Guidelines

Rubric

Rubric		
Criteria	Points	Requirement Description
1. Functionality & Output (40 Points)		
Execution & Logic	20	Program boots in QEMU, clears the screen, and draws a box with the boot messages using exact coordinates given.
Library Integration	10	Correctly links with <code>libos.a</code> and uses <code>putc_to</code> and <code>print_to</code> without modification.
Stability	10	The OS remains stable after execution; no infinite loops (unless intended) or kernel panics.
2. ARMv8 Assembly (box.S) (30 Points)		
Stack Frame Management	10	Properly saves/restores Link Register (x30) and Frame Pointer (x29). Correct stack pointer alignment (16-byte).
Register Preservation	10	Arguments are saved before calling helper functions and restored where necessary to maintain progress bar logic.
Logic Modules	10	Correct implementation of <code>print_progress()</code> function.
3. C Implementation (kernel.c) (10 Points)		
Main Driver	5	<code>main()</code> follows the provided pseudo-code and calls functions in the correct sequence.

Criteria	Points	Requirement Description
Screen Clearing	5	print_progress() uses nested loops and used <code>putc_to()</code> and <code>print_to()</code> to correctly print the message and progress.
4. Build System (Makefile) (15 Points)		
Dependency Graph	10	Correct dependency implementation. Modifying <code>box.S</code> should NOT trigger a re-compile of <code>kernel.c</code> .
Standard Targets	5	Working <code>all</code> , <code>clean</code> , <code>run</code> , and <code>debug</code> targets as specified in the assignment.
5. Packaging & Submission (5 Points)		
Administrative	5	Correct directory naming (yourTnTechId_hw1), clean of binaries, and submitted as a valid <code>.tar.gz</code> or <code>.zip</code> file

Critical Failure Penalties (Automatic 0)

- Failing to implement the Makefile dependency graph correctly, or providing no Makefile at all.
- Writing the `print_progress()` function in C instead of ARMv8 Assembly
- Writing `strlen()` in assembly instead of C
- Not passing parameters according to the specification given in class
- Submitting code that does not compile/link with the provided tools and using the compile command given.

Turn In

You will submit your entire project's directory. Name your project directory as yourTnTechLogin_hw1. For example, if your TnTech email is sclaus42, then your project directory name should be sclause42_hw1. "make clean" your directory first. Then, you can create your tarball with the following command:

```
tar -czf yourTnTechLogin_hw1.tar.gz yourTnTechLogin_hw1/
```

You can also use the Linux file manager to create the compressed file. Just right click on the project directory and you will see the option "Compress...".

(Make sure to substitute yourTnTechLogin for your TnTech login name as you did for your directory name.)



Technical Support

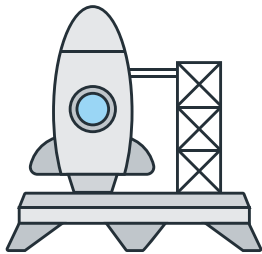
Need help using Canvas Assignments? If so, please review the following guide:

- [Canvas Student Guide - Assignments \(https://design.instructure.com/courses/178/pages/assignments\)](https://design.instructure.com/courses/178/pages/assignments)

Choose a submission type

Upload

More



Choose a file to upload

or

 Webcam Photo

 Canvas Files



<https://canvas.tntech.edu/courses/6565/modules/items/646618>



Assignment
<https://canvas.tntech.edu/courses/6565/modules/items/493482>